Student Performance Analysis System

Project Report & Viva Preparation Guide

A comprehensive explanation for external examiners

Technology: Python, Streamlit, scikit-learn, Pandas, Plotly

Deployment: Streamlit Community Cloud + GitHub

Type: Machine Learning Web Application

1. PROJECT OVERVIEW

The Student Performance Analysis System is a full-stack data analytics and machine learning web application built using Python and Streamlit. It allows educators and administrators to upload student data, explore trends through interactive charts, predict student performance using machine learning models, and generate downloadable reports.

1.1 Main Objectives

- Analyze academic performance of 500+ students across 5 subjects (Math, Physics, Programming, Data Science, Statistics)

- Provide

1.2 Technology Stack

Frontend / UI: Streamlit (Python web framework) - renders dashboard, charts, inputs

2. PROJECT STRUCTURE & FILES EXPLAINED

The project follows a clean MVC-like (Model-View-Controller) structure. All pages are in the views/ folder, business logic is in src/, and static assets like CSS are in assets/.

app.py

Main entry point. Handles routing, authentication gate, sidebar with file upload, and navigation to all 8 pages.

src/auth.py

Authentication module. Contains Sign In and Sign Up forms with SHA-256 password hashing and JSON-based user storage.

src/data_loader.py

Loads CSV or Excel data uploaded by the user, or falls back to the default sample dataset (data/student_performance.csv).

src/preprocessing.py

Cleans the raw data: removes duplicates, fills missing values, applies performance categorization (Low/Average/High based on final_score).

src/models.py

Trains 5 ML classifiers and 1 regressor. Returns predictions, probabilities, confusion matrix, classification report, and feature importances.

src/utils.py

Shared constants (COLORS, SUBJECTS, FEATURE_COLS), CSS injection function, KPI card HTML generator, and Plotly layout settings.

src/eda.py

EDA helper functions: subject statistics, correlation matrix, scatter data, top performers, boxplot data.

views/0_Home.py

Dashboard home/welcome page shown after login.

views/1_Overview.py

Shows 4 KPI cards (Total, Avg, Pass Rate, At-Risk %), Performance Distribution donut chart, Average Score by Category, Top 10 Performers, Attendance Distribution.

views/2_EDA.py

Exploratory Data Analysis: Distribution plots, Correlation Heatmap, Scatter plots, Boxplots, Top/Bottom 10 rankings.

views/3_Student_Analysis.py

Search and select individual students. Shows their profile, radar chart, subject bars, and comparison table.

views/4_Subject_Analysis.py

Subject-level analysis: mean comparison bar chart, pass rate analysis, score distribution histogram,

inter-subject correlation heatmap.

views/5_Prediction.py

Input sliders for student features -> ML model predicts performance category and score. What-If analysis shows impact of improvements.

views/6_Model_Comparison.py

Compares all 5 ML classifiers on Accuracy, Precision, Recall, F1. Shows confusion matrices, feature importance, regression model metrics.

views/7_Reports.py

Generates and downloads student-wise PDF report, full data Excel export, and At-Risk student list with risk level coloring.

assets/style.css

Custom CSS for the dark navy theme, glassmorphism cards, animations, high-contrast dropdown options, button styles.

.streamlit/config.toml

Streamlit configuration: enforces native dark theme, sets primaryColor to cyan (#14B8C4), and text color to white.

data/student_performance.csv

Built-in sample dataset with 500+ students and 15 columns used as default data source.

data/sample_student_records.csv

Ready-made 40-student sample CSV file that can be downloaded and re-uploaded for testing the upload feature.

requirements.txt

Lists all Python package dependencies needed for the project to run (streamlit, pandas, scikit-learn, plotly, fpdf2, etc.).

3. HOW THE SYSTEM WORKS - STEP BY STEP FLOW

Step 1 - User Opens Website

The browser loads the Streamlit app. The app.py first checks if the user is logged in (st.session_state["logged_in"]). If not, it shows the login page from auth.py and stops.

Step 2 - Login / Sign Up

User enters email and password. The password is hashed using SHA-256 (hashlib) and compared with the stored hash in data/users.json. On success, session_state["logged_in"] = True.

Step 3 - Sidebar Loads

After login, the sidebar appears with: Logout button, Data Source upload (CSV/XLSX), and navigation links to all 8 pages.

Step 4 - Data Loading

If user uploads a file, it is read by load_uploaded_data(). It validates that all required columns are present. If not uploaded, the default student_performance.csv is used.

Step 5 - Data Preprocessing

preprocess_data() removes duplicate rows, fills missing values with column means, and assigns a performance_category (Low = score < 50, Average = 50-75, High > 75) to each student.

Step 6 - Page Navigation

Using st.navigation(), the user can navigate to 7 dashboard pages. Each page independently loads data from st.session_state and renders charts.

Step 7 - ML Prediction (Page 5)

The user inputs 7 feature values using sliders. These are scaled using StandardScaler and passed to the best-performing classifier (found by evaluating all 5 models on test data). The predicted category and confidence probabilities are shown.

Step 8 - Report Generation (Page 7)

User selects a student. The system generates a PDF with student details, scores, and performance rating using fpdf2, or an Excel file with full dataset using openpyxl.

4. MACHINE LEARNING MODELS EXPLAINED

The project uses supervised machine learning with labeled data. The target variable is performance_category (Low, Average, High) for classification, and final_score for regression. The data is split 80% training and 20% testing using train_test_split.

4.1 Input Features (7 Features)

- attendance_pct (Attendance %)

- internal_

4.2 Classification Models (predict category: Low/Average/High)

Logistic Regression

A linear model that learns a decision boundary between classes. Fast and interpretable. Uses sigmoid function to output class probabilities.

Decision Tree

Splits data using feature thresholds (e.g., attendance > 75%). Creates a tree of if-else rules. Easy to understand but may overfit.

Random Forest

Builds 100 decision trees on random subsets and takes majority vote. More accurate and robust than a single Decision Tree.

KNN (K-Nearest Neighbors)

Classifies a new student by finding 5 most similar students in the training set and taking majority vote of their categories.

SVM (Support Vector Machine)

Finds the best hyperplane that separates classes with maximum margin. Effective for high-dimensional data.

4.3 Regression Model (predict score)

Linear Regression: Fits a straight line through data points to predict the numerical final_score from the 7 input features. Evaluated using MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), and R2 Score (coefficient of determination).

4.4 How Best Model is Chosen

After training all 5 classifiers, each is evaluated on the test set. The model with the highest Accuracy is automatically selected as the "Best Model". No model is assumed to be best in advance - it is determined purely by performance on this dataset.

5. DATASET EXPLANATION

The system uses a synthetically generated dataset of 515 student records (500 unique + 15 duplicates added for preprocessing demo). The dataset has 15 columns and intentionally includes ~4% missing values to demonstrate data cleaning.

5.1 Column Details

student_id	Unique identifier (STU001, STU002, ...)
name	Student name
gender	Male or Female
attendance_pct	Attendance percentage (40% to 100%)
internal_marks	Internal exam marks out of 50
assignment_marks	Assignment marks out of 25
practical_marks	Practical exam marks out of 25
previous_sem_result	Previous semester score (30-100)
study_hours_per_day	Average daily study hours (0.5-8)
participation_score	Class participation score (1-10)
math_score	Mathematics score (15-100)
physics_score	Physics score (15-100)
programming_score	Programming score (15-100)
data_science_score	Data Science score (15-100)
statistics_score	Statistics score (15-100)
final_score	Average of all 5 subject scores (computed target variable)

5.2 Performance Category Logic

After computing final_score (average of 5 subject scores), each student is categorized as:

- Low: final_score < 50
- Average: 50 <= final_score <= 75
- High: final_score > 75

This is done in preprocessing.py using the get_performance_category() function from utils.py.

6. VIVA QUESTIONS AND ANSWERS

These are commonly asked questions in project viva/demonstration. Read these carefully to confidently explain your project to external examiners.

Q: What is the main objective of your project?

A: The mai
trends usi
generates

Q: What programming language and framework did you use?

A: We use
the interac
generation

Q: What machine learning models did you implement?

A: We imp
SVM. We a
evaluated

Q: How do you select the best model?

A: After tra
with the hig
is data-driv

Q: What is the difference between classification and regression in your project?

A: Classifi
predicts th
Linear Reg

Q: What features (inputs) does the ML model use?

A: The mo
previous_s
predicting

Q: How did you handle missing values in the dataset?

A: In the p
column. W

Q: How is the password stored securely?

A: We use
hashed be
stored hasl

Q: What is StandardScaler and why did you use it?

A: Standar
deviation=1
scale of i
participatio

Q: What is Streamlit and how does it work?

A: Streamli
HTML, CS
Streamlit re

Q: What is st.session_state and why did you use it?

A: st.sessio
and re-run
selections

Q: What charts are used in your project and why?

A: We used
10 Perform
vs Predicte
Importance

Q: What is a confusion matrix?

A: A confu
columns re
show miscl

Q: What is F1-Score?

A: F1-Scor
positive ca
predicted.

Q: What is feature importance?

A: Feature
Forest, it is
importance

Q: How did you deploy the project online?

A: We use
GitHub (sa
automatica

Q: What is the EDA (Exploratory Data Analysis) page?

A: The ED
Distribution
related to e
quartiles, a

Q: What is the What-If Analysis in the Prediction page?

A: What-If
their atten
hypothetica
shows the

Q: What is the At-Risk student feature in the Reports page?

A: At-Risk
final_score
criteria, Me
judgment o

Q: Why did you choose Streamlit over Flask or Django?

A: Streaml
interactive
uploaders,
choice.

7. QUICK REFERENCE SUMMARY

Total Pages	8 (Home, Overview, EDA, Student Analysis, Subject Analysis, Prediction, Model Co

Best of Luck for your Viva! You have built a professional, industry-standard project.